

PROGRAM 17:

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~/OSSP/Program16
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ cp program16.c program17.c
nano program17.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ gcc program17.c -o program17
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ ls
program16  program16.c  program17  program17.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ ./program17
OSSP Program 17 - Job Control
Commands:
  jobs      -> list active jobs
  fg <id>   -> bring job to foreground
  exit      -> exit shell

shell> sleep 30 &
[1] 886 Running sleep 30
shell> sleep 20 &
[2] 891 Running sleep 20
shell> jobs

----- Active Jobs -----
[1] PID=886 Running sleep 30
[2] PID=891 Running sleep 20
-----

shell> sleep 30 &
[3] 892 Running sleep 30
shell> fg 3
Bringing job [3] to foreground...
Command: sleep 30
[3] Completed
[1] Done sleep 30
[2] Done sleep 20
shell> |
```

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#define MAX_JOBS 100
```

```
#define MAX_CMD 256
```

```
typedef struct {
```

```

    int job_id;

    pid_t pid;

    char command[MAX_CMD];

    int running;
} Job;

Job jobs[MAX_JOBS];

int job_count = 0;

int next_job_id = 1;

/* Add background job */
void add_job(pid_t pid, const char *command)
{
    jobs[job_count].job_id = next_job_id++;

    jobs[job_count].pid = pid;

    strncpy(jobs[job_count].command,
            command, MAX_CMD - 1);

    jobs[job_count].command[MAX_CMD - 1] = '\0';

    jobs[job_count].running = 1;

    printf("[%d] %d Running %s\n",
            jobs[job_count].job_id,
            jobs[job_count].pid,
            jobs[job_count].command);

    job_count++;
}

```

```
}
```

```
/* Update job status */
```

```
void update_jobs()
```

```
{
```

```
    for (int i = 0; i < job_count; i++) {
```

```
        if (!jobs[i].running)
```

```
            continue;
```

```
        int status;
```

```
        pid_t result = waitpid(
```

```
            jobs[i].pid,
```

```
            &status,
```

```
            WNOHANG
```

```
        );
```

```
        if (result == jobs[i].pid) {
```

```
            jobs[i].running = 0;
```

```
            printf("[%d] Done %s\n",
```

```
                jobs[i].job_id,
```

```
                jobs[i].command);
```

```
        }
```

```
    }
```

```
}
```

```

/* List jobs */
void show_jobs()
{
    update_jobs();

    printf("\n----- Active Jobs ----- \n");

    for (int i = 0; i < job_count; i++) {

        if (jobs[i].running) {

            printf("[%d] PID=%d Running %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
    }

    printf("----- \n");
}

/* Bring job to foreground */
void foreground_job(int job_id)
{
    for (int i = 0; i < job_count; i++) {

        if (jobs[i].job_id == job_id &&

```

```

jobs[i].running) {

    printf("Bringing job [%d] to foreground...\n",
        job_id);

    printf("Command: %s\n",
        jobs[i].command);

    /*
     * Wait for the selected job
     */
    waitpid(jobs[i].pid, NULL, 0);

    jobs[i].running = 0;

    printf("[%d] Completed\n",
        job_id);

    return;
}

}

printf("Job [%d] not found or already completed.\n",
    job_id);
}

int main()
{

```

```

char input[MAX_CMD];

printf("OSSP Program 17 - Job Control\n");
printf("Commands:\n");
printf(" jobs    -> list active jobs\n");
printf(" fg <id>  -> bring job to foreground\n");
printf(" exit     -> exit shell\n\n");

while (1) {

    update_jobs();

    printf("shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        continue;

    /* Exit */
    if (strcmp(input, "exit") == 0)
        break;

    /* Jobs command */

```

```
if (strcmp(input, "jobs") == 0) {  
    show_jobs();  
    continue;  
}
```

```
/* Foreground command */  
if (strncmp(input, "fg ", 3) == 0) {
```

```
    int job_id = atoi(input + 3);
```

```
    foreground_job(job_id);
```

```
    continue;  
}
```

```
/*  
 * Detect background &  
 */
```

```
int background = 0;
```

```
int len = strlen(input);
```

```
if (len > 0 && input[len - 1] == '&') {
```

```
    background = 1;
```

```
    input[len - 1] = '\0';
```

```

while (len > 1 &&
    input[len - 2] == ' ') {

    input[len - 2] = '\0';
    len--;
}
}

pid_t pid = fork();

if (pid < 0) {

    perror("fork");
    continue;
}

/* Child */
if (pid == 0) {

    char *args[64];
    int argc = 0;

    char *token = strtok(input, " ");

    while (token != NULL &&
        argc < 63) {

        args[argc++] = token;

```

```
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

/* Parent */
else {

    if (background) {

        add_job(pid, input);

    } else {

        waitpid(pid, NULL, 0);
    }
}

}

printf("\nExiting shell...\n");
```

```
/*  
 * Wait for remaining jobs  
 */  
for (int i = 0; i < job_count; i++) {  
  
    if (jobs[i].running) {  
  
        waitpid(jobs[i].pid, NULL, 0);  
    }  
}  
  
return 0;  
}
```